IF2211 STRATEGI ALGORITMA

LAPORAN TUGAS BESAR 2

Disusun oleh:

Kelompok RESERVASI – Kelas 01

Yavie Azka Putra Araly	(13524077)
Angelina Andra Alanna	(13524079)
Kholida Rezki Khoiriah	(13222071)

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI **2**
BAB I
DESKRIPSI TUGAS **4**
1.1. Spesifikasi Wajib 4
1.2. Spesifikasi Bonnus 4
1.3. Konsep Proyeksi dan Overlap 5
BAB II
LANDASAN TEORI **7**
2.1. Document Object Model (DOM) 7
2.2. Depth First Search (DFS) 7
2.3. Perbandingan BFS dan DFS 7
2.4. CSS Selector Matching 7
2.5. Lowest Common Ancestor (LCA) dengan Binary Lifting 7
2.6. Web Scraping 7
BAB III
APLIKASI BFS DAN DFS **8**
3.1. Gambaran Umum Aplikasi 8
3.2. Aristektur Sistem 8
3.3. URL Scraper 8
3.4. Implementasi BFS pada Penelusuran DOM 8
3.5. Implementasi DFS pada Penelusuran DOM 8
3.6. CSS Selector Matching 8
3.7. Animasi Penelurusan 8
3.8. Implementasi LCA Binary Lifting 8
BAB IV
IMPLEMENTASI & PENGUJIAN **9**
4.1. Struktur Proyek 9
4.2. Teknologi yang Digunakan 9
4.3. Pengujian Pencarian 9
4.3.1. Pengujian BFS 9
4.3.2. Struktur DFS 9
4.3.3. Pengujian LCA 9
4.3.4. Pengujian Top N 9
4.4. Struktur Proyek 9
BAB V
KESIMPULAN SARAN **10**
5.1. Kesimpulan 10
5.2. Saran 10
LAMPIRAN **12**

DAFTAR PUSTAKA..... 14

BAB I
DESKRIPSI TUGAS

1.1. Spesifikasi Wajib

Pada tugas ini, mahasiswa diminta untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM.

- Buatlah sebuah aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa Go, C#, Rust, atau Typescript yang mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna
- Secara umum, berikut adalah input yang diharapkan
    - URL website atau memasuki teks HTML sendiri
    - Pilihan algoritma traversal (BFS / DFS)
    - CSS selector
    - Jumlah hasil yang ingin ditampilkan:
        - Top n kemunculan
        - Semua kemunculan
- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut:
    - Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree
    - Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
    - Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
    - Setelah melakukan penelusuran, aplikasi menyimpan sebuah traversal log yang memiliki informasi tahapan penelusuran yang dilakukan.

1.2. Spesifikasi Bonnus

a. (6 poin) [Video] Membuat video tentang aplikasi BFS dan DFS pada penelusuran pohon DOM kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan

menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin menarik video, maka semakin banyak poin yang diberikan.

b. (4 poin) [Deploy] Aplikasi web yang telah dibuat perlu di deploy menggunakan Microsoft Azure Virtual Machine (VM) sehingga aplikasi dapat diakses secara publik melalui internet. Sebagai mahasiswa, terdapat kredit gratis sebesar $100 dari Microsoft untuk dapat digunakan hanya dengan menggunakan email kampus ITB. Karena layanan ini menggunakan sistem pembayaran berbasis penggunaan (pay-as-you-go), penggunaan Virtual Machine akan mengurangi kredit yang dimiliki. Oleh karena itu, disarankan untuk mematikan (stop/deallocate) Virtual Machine ketika tidak digunakan agar kredit Azure tidak cepat habis.
c. (2 Poin) [Docker] Aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend.
d. (6 Poin) [Animasi] Menampilkan animasi penelusuran pohon DOM untuk segala pencarian.
e. (3 Poin) [Multithreading] Mengimplementasikan multithreading pada algoritma BFS dan/atau DFS untuk melakukan traversal pohon DOM secara paralel. Proses traversal dibagi ke beberapa thread/worker sehingga beberapa node dapat diproses secara bersamaan.
f. (3 Poin) [LCA Binary Lifting] Mengimplementasikan algoritma Lowest Common Ancestor (LCA) dengan teknik Binary Lifting pada pohon DOM yang dihasilkan dari proses parsing HTML. LCA digunakan untuk menentukan node leluhur terendah yang sama dari dua node dalam sebuah pohon. Dalam konteks DOM, fitur ini dapat digunakan untuk mengetahui elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu.

BAB II
LANDASAN TEORI

2.1. Document Object Model (DOM)

Document Object Model (DOM) adalah representasi struktur dokumen HTML atau XML dalam bentuk struktur data yang terdiri dari beberapa *node*. DOM membuat isi halaman web tidak hanya dibaca sebagai teks biasa, melainkan juga sebagai elemen-elemen yang saling berhubungan. Setiap *tag* HTML, atribut, dan teks dapat direpresentasikan sebagai bagian dari struktur ini. Dengan DOM, program dapat mengakses, mencari, dan memproses elemen halaman lebih teratur.

Struktur hierarki dalam DOM dibangun berdasarkan hubungan antar *node*. *Node* tertinggi yang menjadi pangkal seluaruh elemen disebut sebagai *root node* (biasanya elemen <html>). Hubungan antar elemen lainnya didefinisikan sebagai *parent* (berada satu tingkat di atas node lain dan memiliki hubungan langsung), *child* (node yang berada satu tingkat di bawah *parent*), dan *sibling* (node yang sejajar dengan *child* di bawah *parent* yang sama).

Berikut contoh kode HTML sebagai ilustrasi.

```
<html>
  <body>
    <h1>Judul</h1>
    <p>Paragraf</p>
  </body>
</html>
```

Kode di atas direpresentasikan menjadi struktur *tree* dengan html sebagai *root*, body sebagai *child* dari html, serta h1 dan p sebagai *siblings* di bawah *parent* body. Representasi *tree* ini bersifat krusial karena memungkinkan penerapan algoritma navigasi graf. Dalam tugas besar ini, data dari hasil *web scraping* diolah ke dalam bentuk DOM *tree* agar dapat ditelusuri menggunakan algoritma DFS dan BFS. Selain itu, untuk mengoptimalkan pencarian hubungan antar elemen yang kompleks, digunakan algoritma *Lowest Common Ancestor* (LCA) dengan teknik *Binary Lifting*. Dengan struktur DOM ini, sistem dapat melakukan ekstraksi data berdasarkan *selector* tertentu dan memvisualisasikan hierarki halaman web dengan efisiensi waktu yang lebih baik.

2.2. Depth First Search (DFS)

Depth First Search (DFS) adalah algoritma penelusuran struktur data *tree* atau *graph* yang bekerja dengan cara menelusuri node sedalam mungkin pada satu cabang terlebih dahulu. Jika cabang tersebut sudah habis (mencapai *leaf node*), algoritma akan kembali ke node sebelumnya (*backtracking*) dan melanjutkan ke cabang lain yang belum dieksplorasi. DFS dapat diimplementasikan menggunakan pendekatan rekursif (memanfaatkan *call stack*) maupun iteratif dengan struktur data *Stack*. DFS sangat efektif untuk mengeksplorasi struktur dokumen web yang memiliki banyak elemen bersarang (*nested*). Dalam sistem ini, DFS digunakan untuk memeriksa setiap node dalam DOM Tree guna mencocokkannya dengan *selector*. Proses penelusuran ini divisualisasikan melalui animasi agar pengguna dapat melihat urutan langkah pencarian secara langsung.

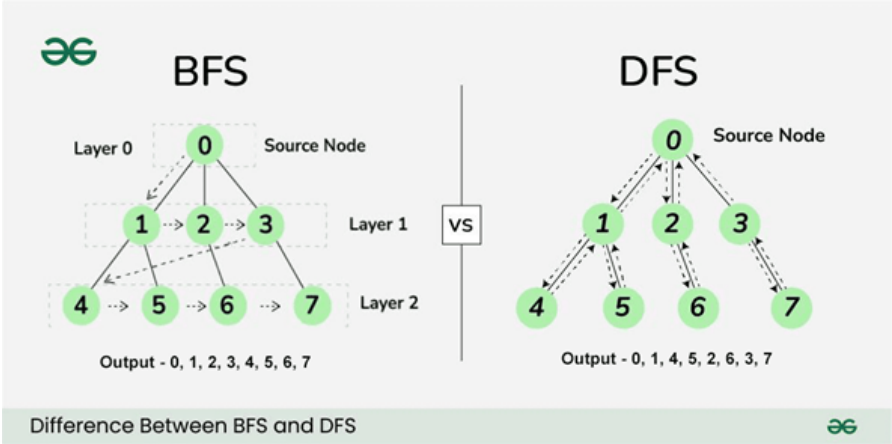
2.3. Perbandingan BFS dan DFS

Breadth First Search (BFS) adalah algoritma traversal yang menelusuri node berdasarkan level atau lapisan. Semua node pada level yang sama akan dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya. Berbeda dengan DFS yang langsung masuk ke satu cabang, BFS bergerak melebar terlebih dahulu. BFS sering digunakan ketika ingin menemukan node yang paling dekat dari root.

BFS umumnya diimplementasikan menggunakan queue. Node yang masuk lebih dulu akan diproses lebih dulu sesuai prinsip FIFO (*First In First Out*). Dengan cara ini, urutan traversal tetap mengikuti level tree. Struktur queue menjadi komponen penting dalam implementasi BFS.

Perbedaan utama BFS dan DFS terletak pada strategi pencariannya. BFS menelusuri node secara melebar per level, sedangkan DFS menelusuri node secara mendalam per cabang. BFS biasanya membutuhkan memori lebih besar karena harus menyimpan banyak node dalam queue. Sebaliknya, DFS cenderung lebih hemat memori, tetapi tidak selalu menemukan node terdekat terlebih dahulu.

Perbedaan dari urutan pencarian pada BFS dan DFS dapat dilihat pada gambar berikut.



Gambar 2.1 Perbedaan BFS dan DFS
(Sumber: https://www.geeksforgeeks.org/dsa/bfs-vs-dfs-binary-tree/)

Dalam tugas besar ini, pengguna dapat memilih BFS atau DFS pada antarmuka sistem. Hasil traversal, jumlah node yang dikunjungi, dan waktu eksekusi dapat dibandingkan dari kedua algoritma tersebut. Hal ini menunjukkan bahwa sistem tidak hanya melakukan pencarian, tetapi juga menampilkan perbedaan pendekatan algoritma traversal.

2.4. CSS Selector Matching

CSS Selector Matching adalah proses mencocokkan elemen HTML dengan pola selector tertentu. Selector merupakan aturan yang biasa digunakan pada CSS untuk memilih elemen yang akan diberi style. Pada tugas besar ini, selector digunakan sebagai kriteria pencarian elemen dalam DOM Tree. Dengan kata lain, selector menentukan node mana yang ingin dicari pengguna.

Selector dapat ditulis dalam berbagai bentuk, seperti tag, class, id, maupun hubungan antar elemen. Setiap bentuk selector memiliki arti yang berbeda dan membutuhkan cara pencocokan yang berbeda pula. Program akan memeriksa setiap node yang dikunjungi untuk menentukan apakah node tersebut memenuhi aturan selector. Jika sesuai, node akan dimasukkan ke hasil pencarian.

Beberapa contoh selector ditunjukkan pada tabel berikut:

Selector	Fungsi
`h1`	Memilih semua elemen <h1>
`p`	Memilih semua elemen <p>
`.card`	Memilih elemen dengan class card
`#main`	Memilih elemen dengan id main
`div p`	Memilih elemen p di dalam div
`div > p`	Memilih child langsung p dari div

Dalam tugas besar ini, BFS atau DFS berfungsi untuk menelusuri node satu per satu, sedangkan selector matching berfungsi untuk menentukan node yang cocok. Kedua proses tersebut berjalan bersama selama pencarian dilakukan. Tanpa selector matching, sistem hanya menelusuri tree tanpa mengetahui elemen target yang dicari.

2.5. Lowest Common Ancestor (LCA) dengan Binary Lifting

Lowest Common Ancestor (LCA) adalah node leluhur terdalam yang menjadi ancestor bersama dari dua node pada tree. Leluhur terdalam berarti node tersebut adalah ancestor yang paling dekat dengan kedua node yang dibandingkan. Konsep ini sering digunakan untuk menganalisis hubungan antar node dalam struktur hierarki. Pada DOM Tree, LCA dapat menunjukkan elemen induk bersama dari dua elemen HTML.

Untuk tree berukuran besar, pencarian LCA secara langsung dapat memerlukan waktu lebih lama. Oleh karena itu, digunakan metode Binary Lifting**,** yaitu teknik preprocessing yang menyimpan ancestor node pada jarak pangkat dua. Contohnya adalah parent ke-1, ke-2, ke-4, ke-8, dan seterusnya. Dengan teknik ini, pencarian ancestor dapat dilakukan lebih efisien.

Dalam tugas besar ini, LCA digunakan sebagai fitur bonus untuk mencari hubungan antara dua node pada DOM Tree. Hasil pencarian divisualisasikan melalui penandaan node ancestor dan jalur terkait pada antarmuka. Fitur ini menambah kemampuan sistem dalam menganalisis struktur DOM selain hanya mencari elemen.

2.6.Web Scraping

Web Scraping adalah teknik mengambil data dari halaman web secara otomatis menggunakan program. Program akan mengakses sebuah URL, membaca isi HTML halaman tersebut, lalu mengekstrak informasi yang dibutuhkan. Web scraping banyak digunakan untuk pengumpulan data, monitoring, maupun analisis konten website. Pada tugas besar ini, data utama yang diambil adalah dokumen HTML dari halaman web.

Secara umum, proses web scraping dimulai dengan mengirim request ke server website. Setelah server memberikan respons, program menerima isi halaman dalam bentuk HTML. HTML tersebut kemudian diubah menjadi DOM Tree agar dapat ditelusuri menggunakan algoritma traversal. Dengan demikian, scraping merupakan tahap awal sebelum proses pencarian dilakukan.

Sebagai contoh, jika pengguna memasukkan URL https://example.com, sistem akan mengambil HTML dari halaman tersebut. Setelah itu, HTML diproses menjadi tree dan elemen seperti <h1> atau <p> dapat dicari menggunakan selector. Hasil akhirnya kemudian ditampilkan kembali pada antarmuka pengguna.

Dalam tugas besar ini, web scraping menghubungkan sumber data dari internet dengan sistem traversal yang dibuat. Tanpa proses ini, program hanya dapat bekerja pada HTML yang dimasukkan secara manual. Oleh karena itu, web scraping menjadi komponen penting agar aplikasi dapat digunakan pada website nyata.

Secara garis besar, seluruh konsep di atas terkait dalam satu alur kerja sebagai berikut.

1. Sistem mengambil data melalui Web Scraping**.**
2. HTML di-*parsing* menjadi struktur DOM Tree**.**
3. Sistem melakukan *preprocessing* untuk fitur LCA Binary Lifting**.**
4. Pengguna memilih algoritma (BFS atau DFS) dan memasukkan CSS Selector**.**
5. Selama traversal, dilakukan Selector Matching pada setiap node.
6. Hasil akhir ditampilkan bersama visualisasi jalur, log, dan statistik performa.

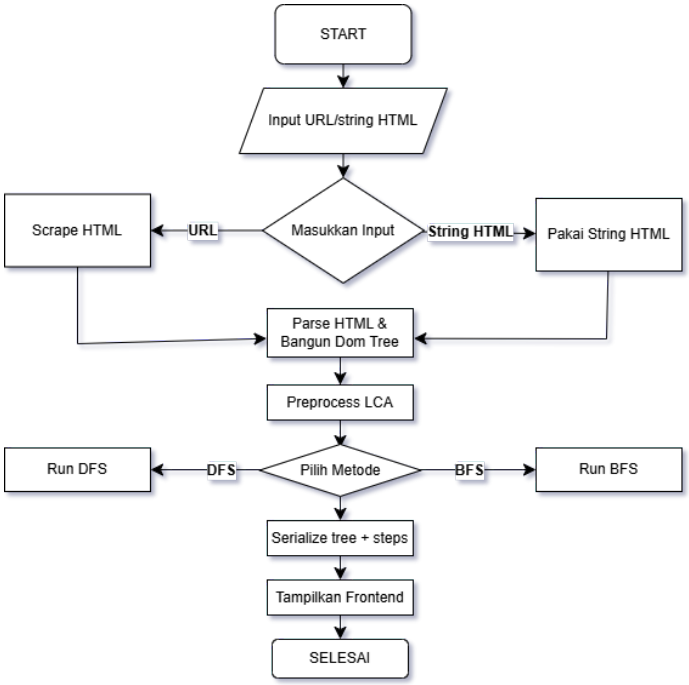
BAB III
APLIKASI BFS DAN DFS

3.1.Gambaran Umum Sistem

Aplikasi DOM Transversal Engine dirancang sebagai sistem berbasis web yang memungkinkan pengguna menelusuri struktur pohon HTML menggunakan algoritma BFS atau DFS berdasarkan CSS Selector. Sistem ini terdiri dari dua lapisan utama yaitu frontend berbasis React yang menangani interaksi antara pengguna dan visualisasi pohon DOM, serta backend berbasis Next.js App Router yang menjalankan scraping, parsing, traversal, dan matching.

Secara garis besar, alur kerja sistem mengikuti urutan:

1. Pengguna memberikan input
2. User dapat memasukkan url website dan string HTML,
   bila user memasukkan url website sistem akan terlebih dahulu melakukan scraping untuk mendapatkan string html.
3. User dapat memasukkan metode transversal (DFS atau BFS) dan memilih untuk menampilkan semua match, atau topN match. User juga dapat mencari LCA antara 2 node.
4. Hasilnya lalu ditampilkan sebagai visualisasi interaktif dengan animasi step-by-step.
   (Gambar 3.1 menunjukkan alur sistem secara keseluruhan)



Gambar 3.1 Alur Sistem Keseluruhan

3.2.Implementasi BFS pada Penelusuran DOM

Pencarian elemen menggunakan algoritma Breadth-First Search (BFS) diimplementasikan untuk melakukan penelusuran pohon DOM secara melebar atau level-demi-level. Implementasi algoritma ini terdapat pada class TreeTraversal di mana proses penelusuran dipisahkan dari logika pencocokan (matching), sehingga memberikan modularitas yang baik.

Algoritma BFS memanfaatkan struktur data antrean (queue) untuk menyimpan node yang akan dikunjungi. Dalam implementasi menggunakan TypeScript, struktur data ini direpresentasikan dengan array DOMNode[]. Berikut adalah tahapan proses implementasi penelusuran BFS pada sistem:

1. Proses dimulai dengan memasukkan node root (akar) dari pohon DOM ke dalam sebuah antrean tunggal.
2. Sistem akan menjalankan iterasi (perulangan) selama kondisi antrean tidak kosong.
3. Pada setiap iterasi, node yang berada di posisi paling depan dalam antrean akan dikeluarkan menggunakan operasi shift(). Hal ini memastikan prinsip First-In-First-Out (FIFO) tetap terjaga selama penelusuran.
4. Node yang diekstrak kemudian diproses dengan mengirimkannya ke fungsi callback onVisit(). Fungsi inilah yang menjadi jembatan antara traversal pohon

dengan evaluasi kondisi CSS Selector Matching. Di dalam callback ini, sistem mengecek kecocokan node dengan elemen yang dicari tanpa mengganggu alur traversal utama.

5. Jika node saat ini memiliki turunan (direpresentasikan oleh properti children), maka semua child node tersebut akan dimasukkan satu per satu ke bagian belakang antrean menggunakan operasi push().

Karena penggunaan mekanisme callback ini, sistem dapat menangani pencarian elemen (baik menampilkan semua kemunculan maupun batas iterasi untuk pencarian Top N kemunculan) dengan lebih fleksibel tanpa perlu mengubah algoritma dasar pencariannya.

3.3.Implementasi DFS pada Penelusuran DOM

Berbeda dengan BFS yang menelusuri pohon secara melebar, algoritma Depth-First Search (DFS) diimplementasikan untuk melakukan penelusuran pohon DOM secara mendalam. Implementasi ini menggunakan pendekatan rekursif yang secara alami memanfaatkan struktur data stack (tumpukan) melalui call stack pada mesin eksekusi TypeScript.

Berdasarkan implementasi pada sistem, berikut adalah karakteristik dan alur kerja dari algoritma DFS tersebut:

1. Algoritma mengunjungi node saat ini terlebih dahulu sebelum menelusuri anak-anaknya. Hal ini dipastikan dengan pemanggilan fungsi callback onVisit(root) di awal fungsi sebelum masuk ke tahap rekursi.
2. Untuk setiap node yang sedang diproses, algoritma akan segera melakukan pemanggilan rekursif ke anak pertamanya, lalu ke cucu, dan seterusnya hingga mencapai leaf node (simpul daun) paling dalam pada cabang tersebut sebelum kembali (backtrack) untuk menelusuri sibling (saudara simpul) berikutnya.
3. Mekanisme Iterasi: Algoritma melakukan iterasi terhadap array children yang ada pada setiap objek DOMNode. Di dalam iterasi tersebut, fungsi this.dfs(child, onVisit) dipanggil kembali secara mandiri untuk setiap elemen anak.
4. Abstraksi Logika: Sama halnya dengan implementasi BFS, logika untuk mengevaluasi apakah suatu simpul cocok dengan CSS Selector atau untuk mencatat tahapan penelusuran ke dalam traversal log diabstraksikan sepenuhnya melalui parameter onVisit. Dengan cara ini, algoritma DFS tetap berfungsi murni sebagai penjelajah struktur data pohon.

Karakteristik DFS yang mendahului kedalaman ini memberikan pola penelusuran yang berbeda dibandingkan BFS, di mana sistem akan menelusuri satu cabang hingga tuntas sebelum

berpindah ke cabang lainnya. Hal ini memungkinkan pengguna untuk membandingkan efisiensi kedua algoritma dalam menemukan elemen yang sama pada struktur DOM yang kompleks melalui visualisasi yang tersedia di aplikasi.

BAB IV
IMPLEMENTASI & PENGUJIAN

4.1.Struktur Proyek

Implementasi sistem ini dibagi menjadi beberapa modul fungsional di sisi backend untuk memastikan kode yang bersih, mudah dipelihara, dan memiliki tanggung jawab yang terisolasi (separation of concerns). Seluruh kode program dikembangkan menggunakan TypeScript untuk menjamin keamanan tipe data.

Berikut adalah rincian struktur file pada direktori backend beserta fungsinya masing-masing:

- route.ts: Bertindak sebagai API gateway dan pengontrol utama yang menjembatani interaksi antara antarmuka pengguna (frontend) dengan logika inti backend. Modul ini menerima permintaan HTTP POST yang berisi parameter pencarian pengguna (seperti URL, string HTML, metode algoritma, dan batas hasil), lalu mengoordinasikan eksekusi pemrosesan dari awal hingga akhir. Selain menjalankan algoritma utama dan LCA, modul ini juga bertanggung jawab mengubah representasi pohon DOM menjadi struktur datar (flat tree) serta menghasilkan traversal log secara terstruktur untuk divisualisasikan oleh frontend.
- scraper.ts: Modul yang bertanggung jawab untuk mengambil source code HTML dari URL yang diberikan oleh pengguna menggunakan pustaka axios. Modul ini juga dilengkapi dengan konfigurasi user-agent untuk mensimulasikan permintaan dari peramban.
- domtree.ts: Berisi definisi struktur data utama, yaitu class DOMNode. File ini mendefinisikan atribut-atribut node seperti tag, teks, serta pointer relasi antar simpul (parent, children, siblings).
- lexer.ts: Berisi logika HTML Tokenizer yang memecah string HTML mentah menjadi kumpulan token seperti StartTag, EndTag, dan Text.
- parser.ts: Mengimplementasikan DOMParser yang bertugas menyusun token-token dari lexer menjadi struktur pohon DOM yang utuh menggunakan pendekatan stack.
- selector-css.ts: Berisi lexer khusus untuk memproses string CSS Selector menjadi kumpulan token selector (seperti ID, class, atau combinator).
- selector-parser.ts: Bertugas untuk memproses token selector menjadi urutan langkah pencarian (SelectorSteps) yang terstruktur.

- selector-matcher.ts: Mengimplementasikan logika pencocokan (matching) antara DOM Node dengan CSS Selector menggunakan strategi right-to-left sebagaimana standar mesin peramban modern.
- traversal.ts: Berisi implementasi utama dari algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) yang digunakan untuk menjelajahi pohon DOM.

4.2.Teknologi yang Digunakan

Berikut adalah daftar teknologi utama yang digunakan:

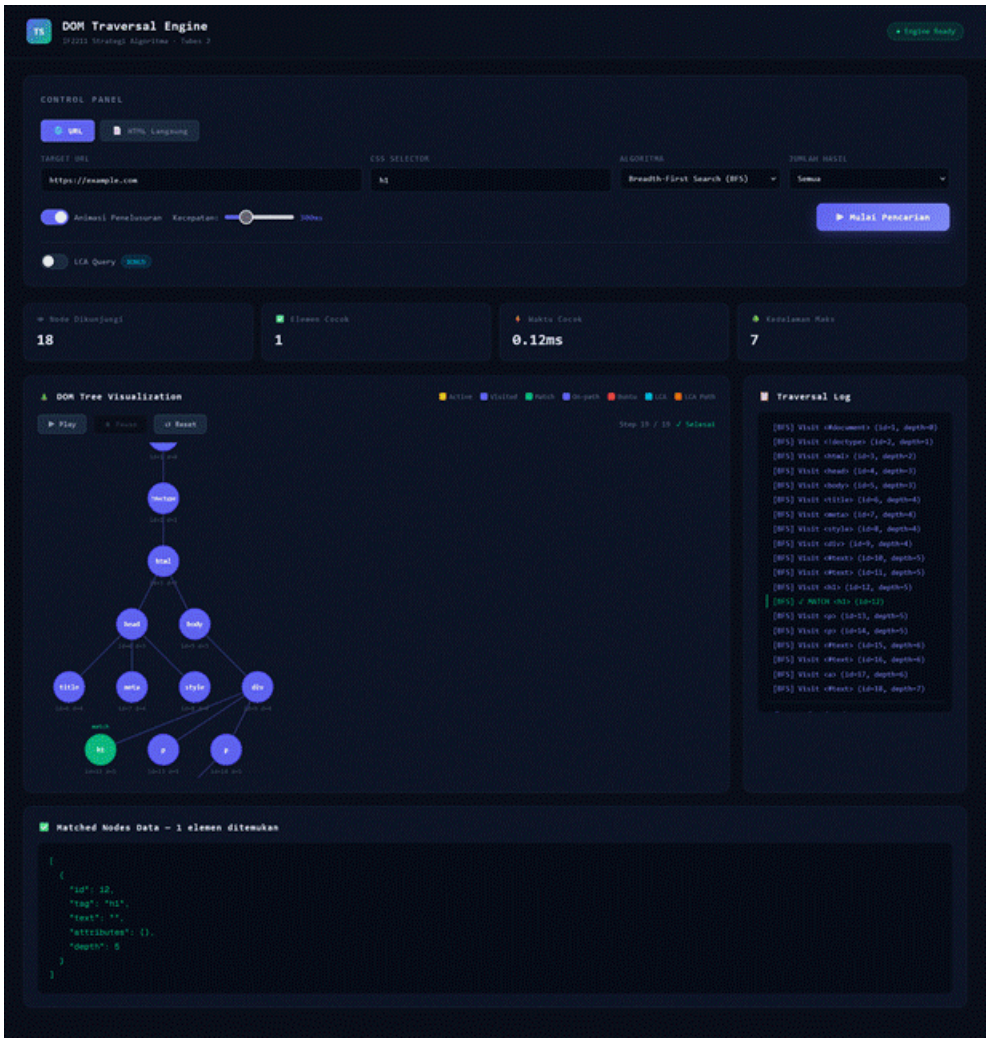
- TypeScript: Digunakan sebagai bahasa pemrograman utama untuk seluruh ekosistem proyek (frontend dan backend). Fitur static typing pada TypeScript sangat membantu dalam meminimalkan bug selama pengembangan, terutama saat merancang dan mengelola struktur data graf dan pohon kompleks seperti DOMNode.
- Next.js (App Router): Bertindak sebagai framework utama aplikasi. Pada sisi backend, Next.js dimanfaatkan melalui fitur API Routes (route.ts) untuk menangani logika pemrosesan yang berat (scraping, parsing, traversal) dan menyediakan REST API.
- React: Digunakan pada lapisan frontend untuk membangun komponen antarmuka pengguna yang dinamis dan interaktif, termasuk menangani visualisasi dan animasi step-by-step dari pohon DOM berdasarkan balasan dari backend.
- Axios: Merupakan library HTTP Client berbasis Promise yang diimplementasikan pada modul backend. Axios digunakan secara spesifik untuk melakukan web scraping, yaitu mengambil teks HTML dari URL eksternal secara asinkron dengan konfigurasi user-agent kustom untuk menghindari pemblokiran.
- Docker: Digunakan dalam fase infrastruktur untuk mengemas (containerize) aplikasi secara utuh. Penggunaan Docker memastikan aplikasi dapat dijalankan secara konsisten di berbagai environment pengembangan maupun produksi.
- Microsoft Azure Virtual Machine (VM): Layanan infrastruktur komputasi awan (IaaS) dari Microsoft yang digunakan sebagai server hosting. Aplikasi di-deploy ke dalam Azure VM agar fungsionalitas DOM Traversal Engine dapat diakses secara publik melalui jaringan internet.

4.3.Pengujian Pencarian

4.3.1. Pengujian BFS

Pengujian ini dilakukan untuk memastikan algoritma Breadth-First Search (BFS) dapat menelusuri struktur DOM dan menemukan elemen yang sesuai dengan CSS selector. Pengujian dilakukan pada dua jenis input, yaitu URL website dan HTML yang dimasukkan secara langsung.

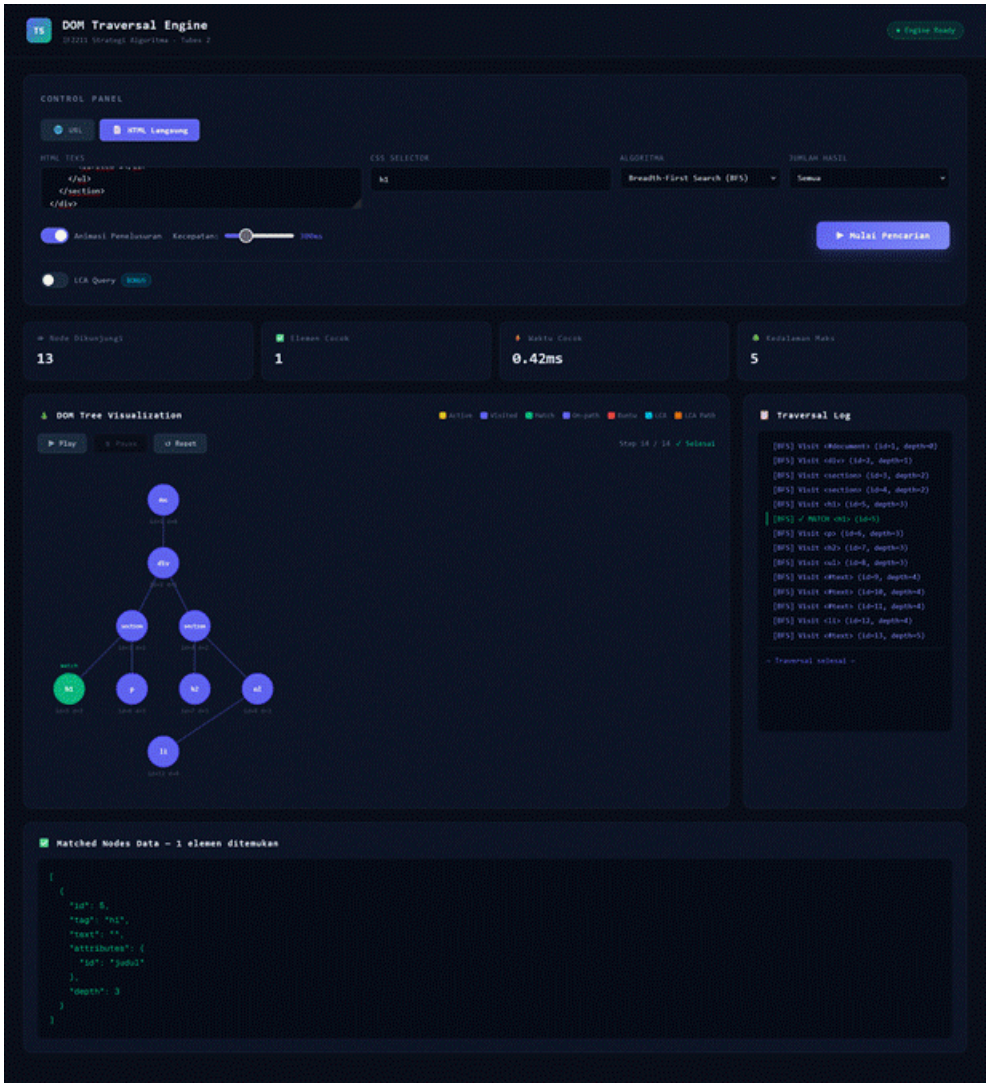
Pada skenario pertama, input berupa URL https://example.com dengan selector h1. Sistem berhasil mengambil dokumen HTML dari website, membentuk DOM Tree, lalu melakukan traversal menggunakan BFS. Hasil pengujian menunjukkan elemen <h1> berhasil ditemukan pada node id-12 dengan kedalaman 5. Jumlah node yang dikunjungi sebanyak 18 node dengan waktu pencarian 1.35 ms.



Gambar 4.1 Hasil Pengujian BFS menggunakan URL https://example.com

Pada skenario kedua, input berupa HTML yang dituliskan langsung ke sistem dengan selector h1. Sistem berhasil memproses HTML tanpa perlu melakukan scraping dari website eksternal. Hasil pengujian menunjukkan elemen <h1> ditemukan pada node id-5 dengan atribut id="judul" dan kedalaman 3. Jumlah node yang dikunjungi sebanyak 13 node dengan waktu pencarian 0.42 ms.

Traversal log pada kedua skenario menunjukkan bahwa BFS mengunjungi node berdasarkan level, dimulai dari root menuju node-node pada level berikutnya. Hal ini sesuai dengan karakteristik algoritma Breadth-First Search.

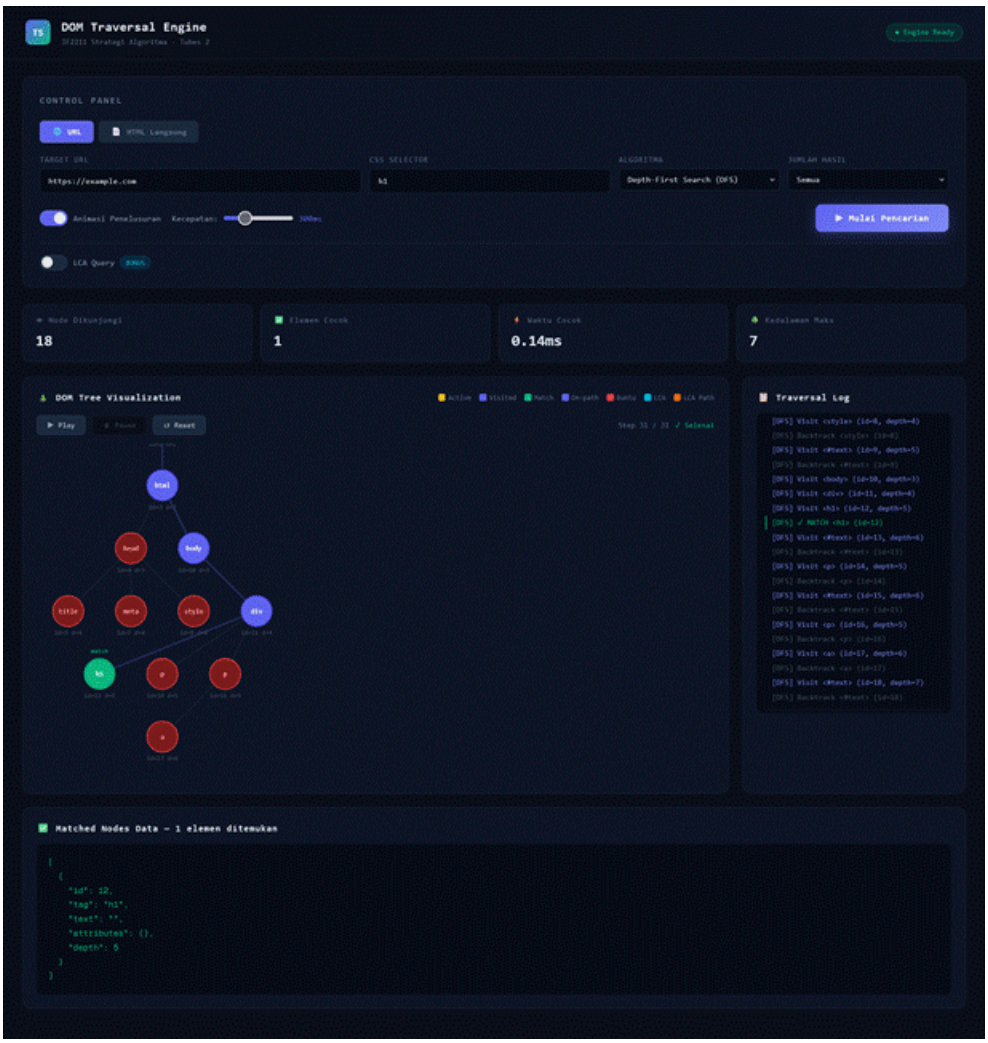


Gambar 4.2 Hasil Pengujian BFS menggunakan input HTML langsung

4.3.2. Pengujian DFS

Pengujian ini dilakukan untuk memastikan algoritma Depth-First Search (DFS) dapat menelusuri DOM Tree dan menemukan elemen yang sesuai dengan CSS selector. Pengujian dilakukan pada tiga jenis input, yaitu URL website sederhana, website nyata, dan HTML yang dimasukkan secara langsung.

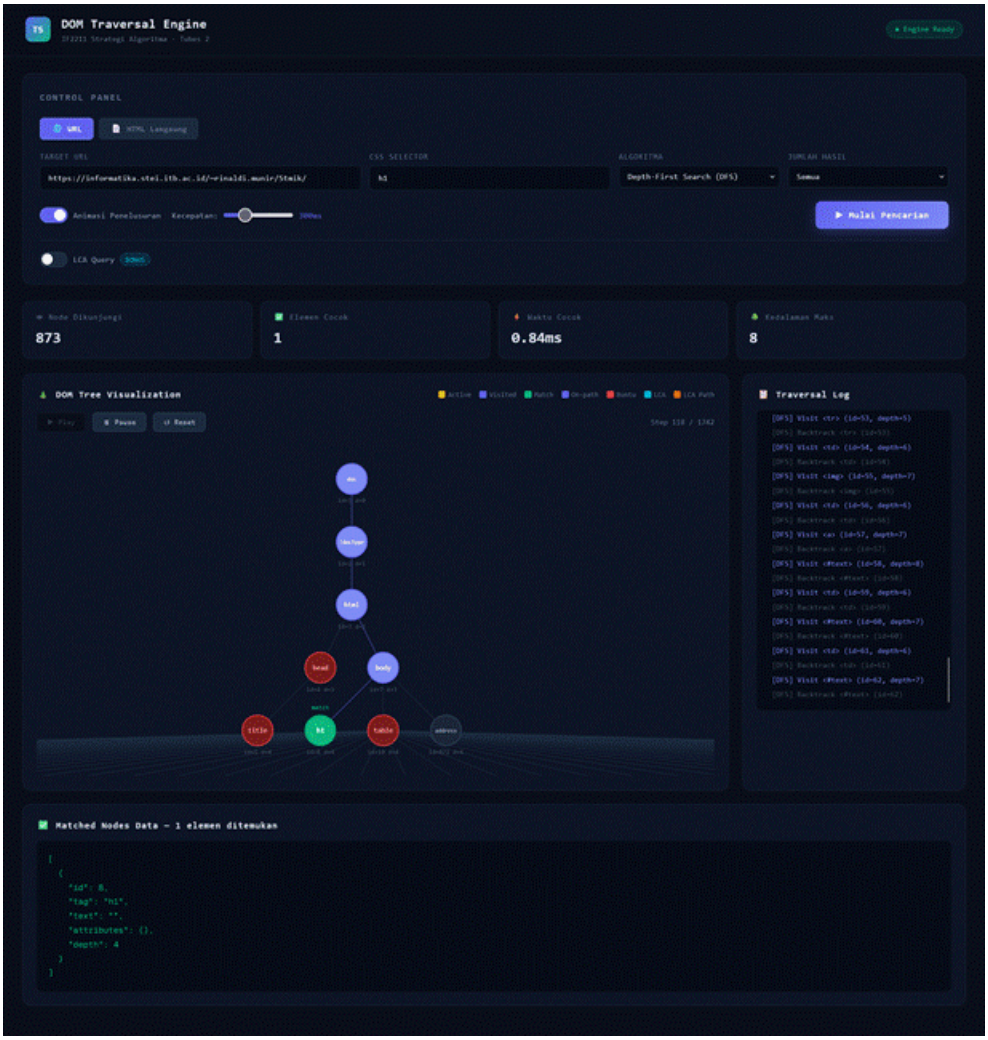
Pada skenario pertama, input menggunakan URL https://example.com dengan selector h1. Sistem berhasil melakukan traversal dan menemukan elemen <h1> pada node id-12 dengan kedalaman 5. Jumlah node yang dikunjungi sebanyak 18 node dengan waktu pencarian 0.14 ms.



Gambar 4.3 Hasil Pengujian DFS menggunakan URL https://example.com

Pada skenario kedua, input menggunakan website dosen pengampu dengan selector h1. Sistem berhasil memproses halaman web dengan struktur DOM yang lebih kompleks dibandingkan website sederhana. Elemen <h1> ditemukan pada node id-8 dengan kedalaman 4. Jumlah node yang dikunjungi sebanyak 873 node dengan waktu pencarian 0.84 ms.

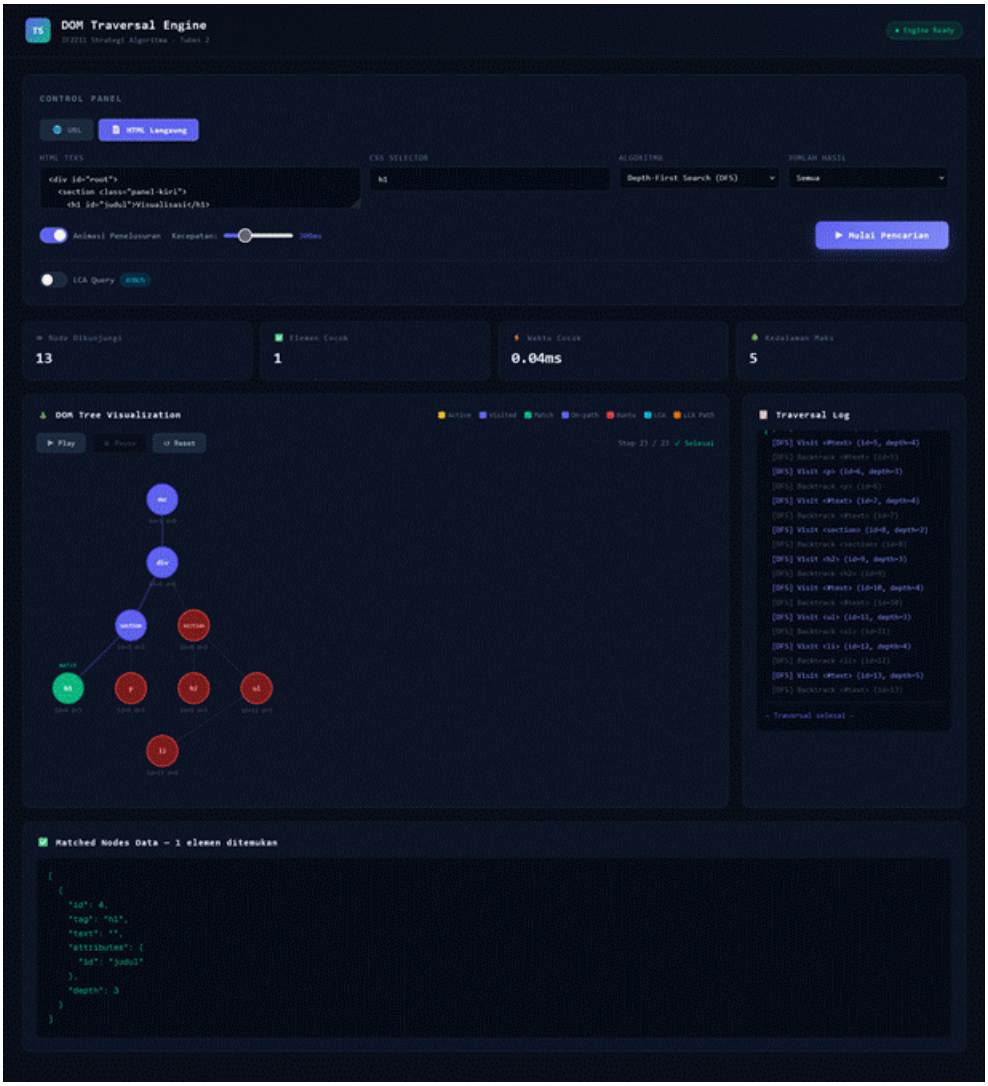
Hasil ini menunjukkan bahwa sistem tetap dapat berjalan pada halaman web nyata dengan jumlah node yang lebih banyak.



Gambar 4.4 Hasil Pengujian DFS menggunakan website dosen pengampu

Pada skenario kedua, input menggunakan website dosen pengampu dengan selector h1. Sistem berhasil memproses halaman web dengan struktur DOM yang lebih kompleks dibandingkan website sederhana. Elemen <h1> ditemukan pada node id-8 dengan kedalaman 4. Jumlah node yang dikunjungi sebanyak 873 node dengan waktu pencarian 0.84 ms.

Hasil ini menunjukkan bahwa sistem tetap dapat berjalan pada halaman web nyata dengan jumlah node yang lebih banyak.



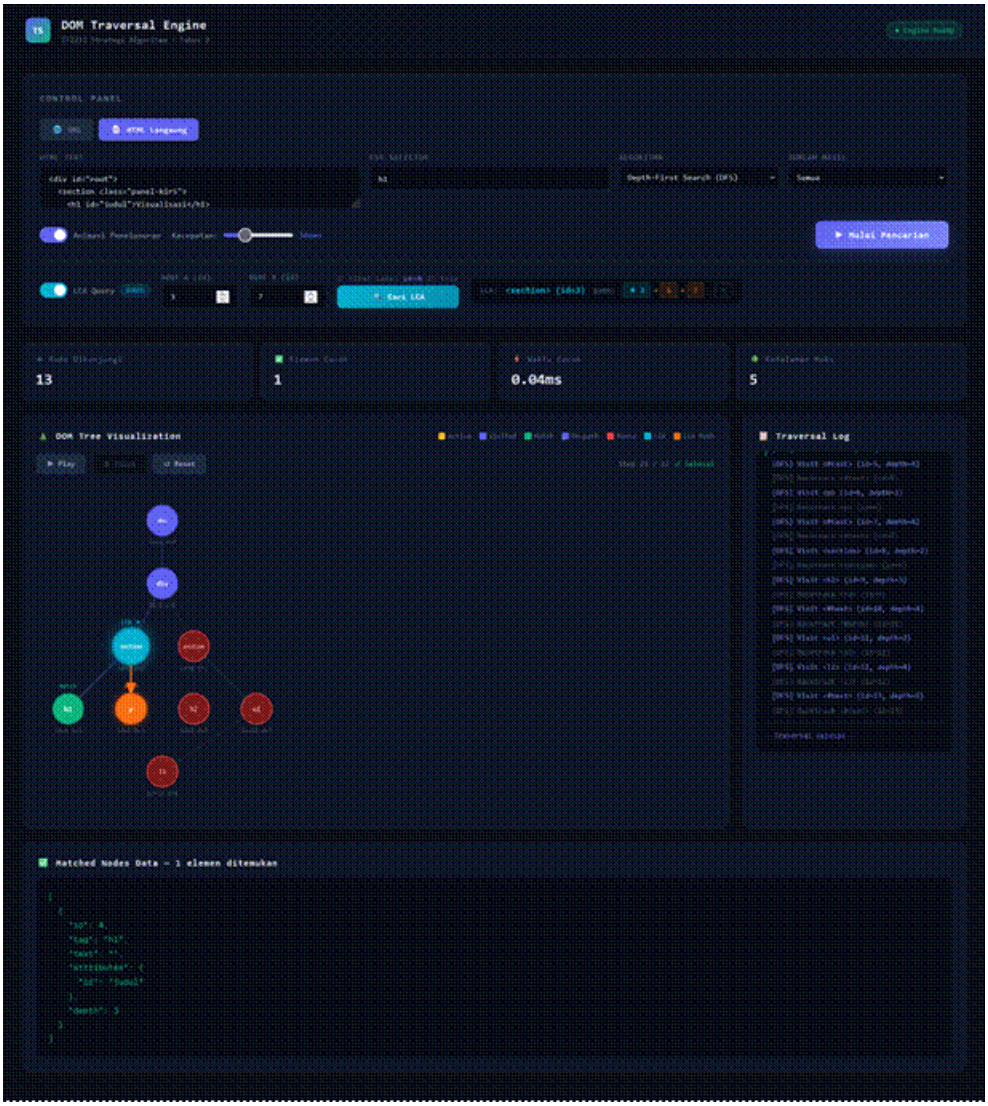
Gambar 4.5 Hasil Pengujian DFS menggunakan input HTML langsung

4.3.3. Pengujian LCA

Pengujian ini dilakukan untuk memastikan fitur Lowest Common Ancestor (LCA) dapat menentukan ancestor bersama terdekat dari dua node pada DOM Tree. Pengujian menggunakan input HTML yang dimasukkan langsung ke sistem agar struktur tree lebih mudah diamati.

Pada pengujian ini dipilih dua node, yaitu node id-3 dan node id-7. Setelah proses query dijalankan, sistem menampilkan hasil bahwa ancestor bersama terdekat dari kedua node tersebut adalah elemen <section> dengan id-3.

Selain menampilkan hasil node ancestor, sistem juga memberikan visualisasi jalur dari node yang dipilih menuju node LCA. Hal ini menunjukkan bahwa fitur LCA telah berjalan sesuai dengan konsep tree traversal dan analisis hubungan antar node.

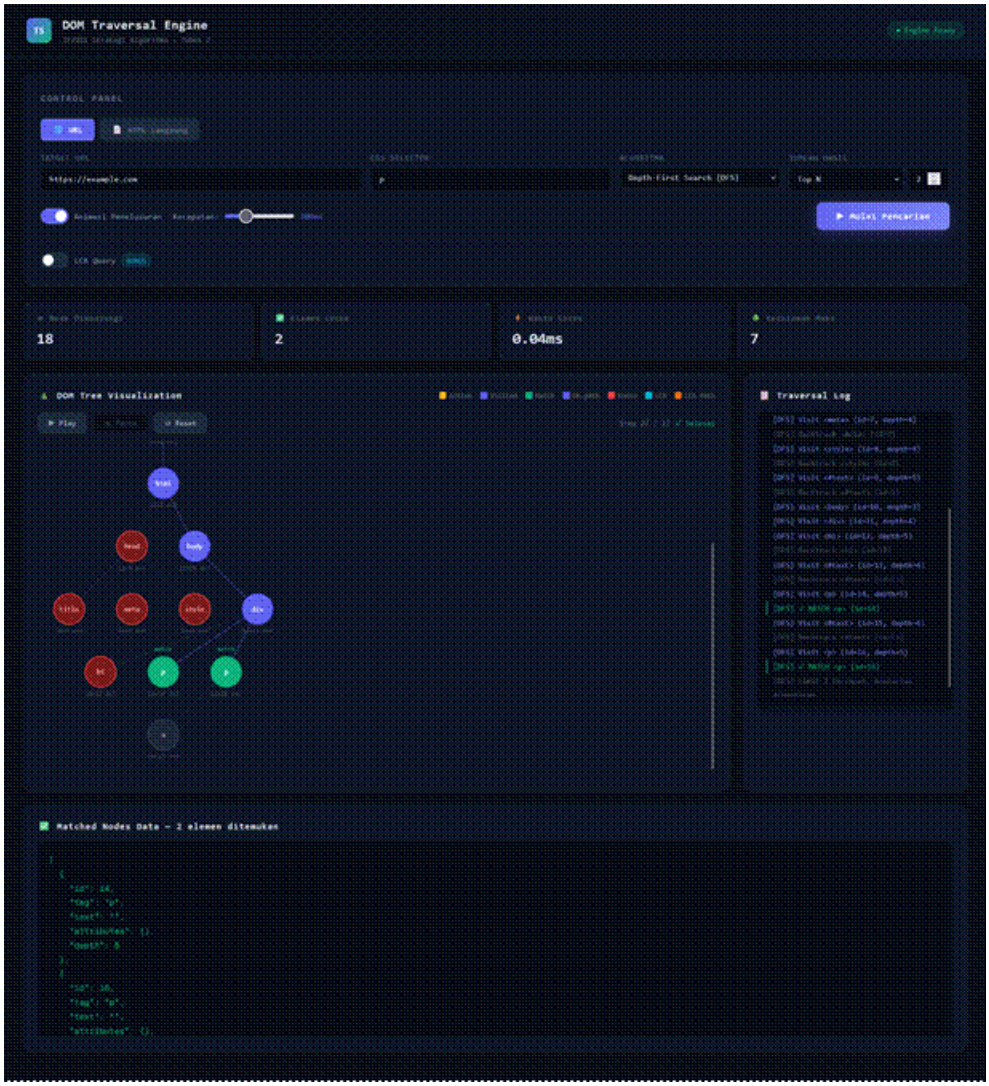


Gambar 4.6 Hasil Pengujian Fitur Lowest Common Ancestor (LCA)

4.3.4. Pengujian Top N

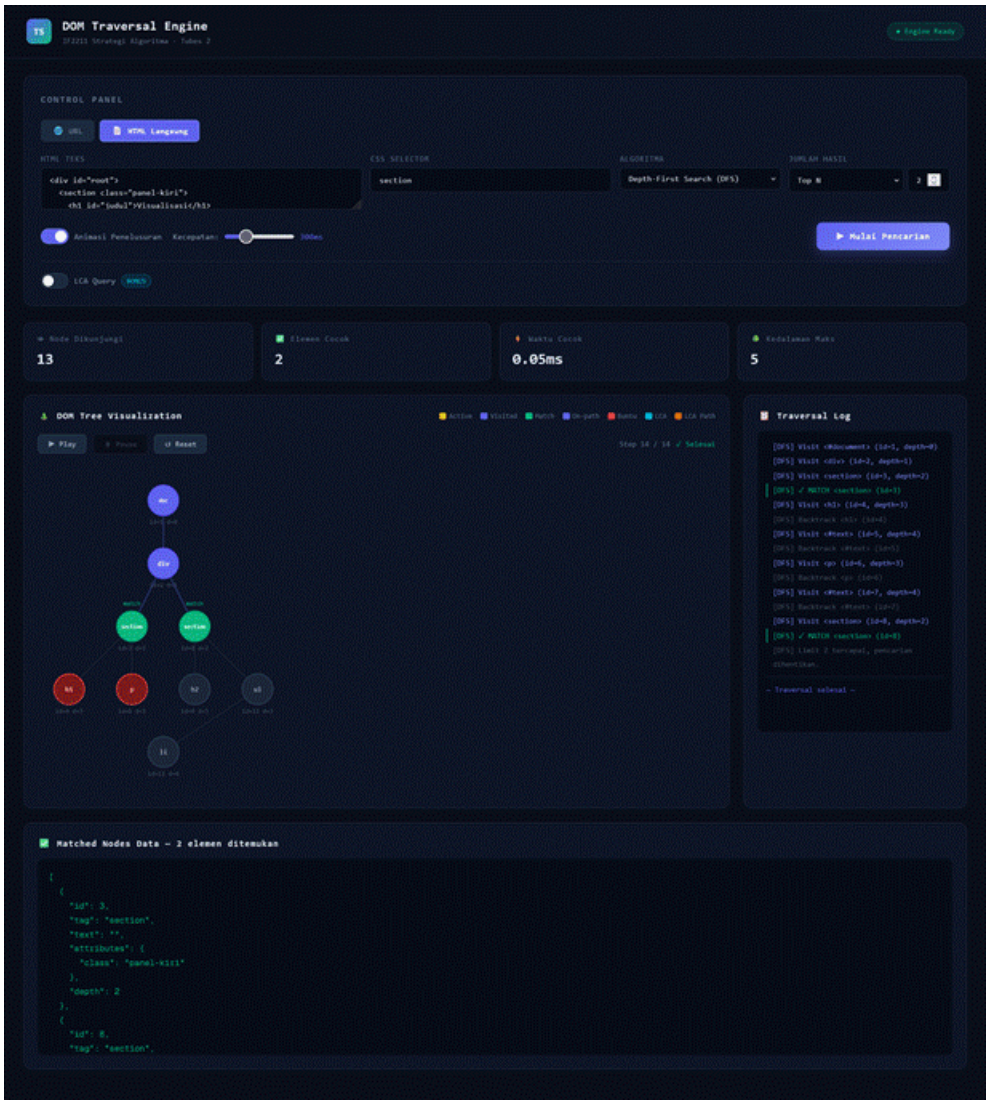
Pengujian ini dilakukan untuk memastikan fitur Top N dapat membatasi jumlah elemen hasil pencarian sekaligus menghentikan proses traversal setelah jumlah hasil yang diminta tercapai. Sistem diuji menggunakan dua jenis input, yaitu URL website dan HTML yang dimasukkan langsung ke aplikasi. Dengan pengujian ini dapat diamati apakah sistem tidak hanya menampilkan hasil sesuai batas, tetapi juga melakukan pencarian secara lebih efisien.

Pada pengujian pertama digunakan URL https://example.com dengan selector p, algoritma DFS, dan jumlah hasil Top N = 2. Setelah proses dijalankan, sistem menemukan dua elemen <p> yang sesuai selector. Pada traversal log juga terlihat keterangan bahwa limit 2 tercapai dan pencarian dihentikan. Hal ini menunjukkan bahwa sistem berhasil menerapkan pembatasan hasil sekaligus early stopping pada proses traversal.



Gambar 4.7 Hasil Pengujian Fitur Top N menggunakan URL https://example.com

Pada pengujian kedua digunakan HTML manual dengan selector section, algoritma DFS, dan jumlah hasil Top N = 2. Sistem menampilkan dua elemen <section> yang terdapat pada struktur HTML. Traversal log kembali menunjukkan bahwa proses pencarian dihentikan setelah dua elemen ditemukan, sehingga node lain yang tidak diperlukan tidak diproses lebih lanjut.



Gambar 4.8 Hasil Pengujian Top N menggunakan input HTML langsung

Berdasarkan kedua pengujian tersebut, fitur Top N telah berjalan dengan baik karena mampu menampilkan hasil sesuai jumlah yang diminta dan menghentikan traversal setelah batas tercapai. Mekanisme ini membuat proses pencarian menjadi lebih efisien, terutama pada struktur DOM yang besar dan memiliki banyak elemen yang cocok.

BAB V
KESIMPULAN SARAN

5.1. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Algoritma BFS dan DFS berhasil diimplementasikan untuk melakukan penelusuran elemen pada pohon DOM berdasarkan CSS selector. Kedua algoritma memberi hasil yang sesuai dan dapat dibedakan karakteristiknya melalui visualisasi animasi.
2. CSS Selector matching dengan pendekatan right to left berhasil mendukung semua jenis selector dasar (tag, class, id, universal) dan combinator (>, spasi, +, ~) sesuai dengan spesifikasi tugas.
3. Fitur animasi step-by-step berhasil memberikan pemahaman yang lebih jelas tentang cara kerja algoritma BFS dan DFS.
4. Fitur LCA dengan binary lifting berhasil diimplementasikan dan dapat menentukan ancestor bersama dari dua Node DOM dengan kompleksitas algoritma O(log N) per query.
5. Fitur Top N dengan early stop berhasil mengoptimalisasikan transversal sehingga pencarian berhenti tepat saat jumlah hasil yang diinginkan telah tercapai, tanpa perlu menelusuri seluruh pohon.

5.2. Saran

Beberapa hal yang bisa dikembangkan lebih lanjut pada versi berikutnya.

1. Dukungan CSS selector dapat diperluas agar sistem mampu menangani selector yang lebih kompleks dan variasi kombinasi selector yang lebih banyak. Dengan begitu, proses pencarian elemen akan menjadi lebih fleksibel dan hasilnya semakin mendekati cara kerja browser sebenarnya.
2. Tampilan visualisasi DOM Tree masih dapat ditingkatkan, terutama untuk halaman web dengan jumlah node yang besar. Misalnya dengan menambahkan fitur zoom, pencarian node tertentu, atau menyembunyikan cabang yang tidak diperlukan agar struktur tree lebih mudah dibaca.
3. Fitur bonus yang sudah dibuat masih dapat terus dikembangkan, seperti peningkatan animasi traversal, optimasi performa program, serta penyempurnaan proses deployment agar aplikasi lebih stabil dan mudah diakses.

LAMPIRAN

Tambahkan tabel berikut yang diisi checklist (✓) pada bagian lampiran laporan Anda.

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS Selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pada pohon	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Link Video Youtube: https://youtu.be/SQOnylKzpDQ

Link Repositori Gituhub: https://github.com/YavieAzka/Tubes2_RESERVASI/

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Yavie Azka Putra Araly

Angelina Andra Alanna

Kholida Rezki Khoiriah

DAFTAR PUSTAKA

1. **L04 : LCA Part 2 : Binary Lifting | Graph Theory Part 2 | CodeNCode :** https://youtu.be/k6AQNBwGgbQ?si=Hk2KKzwbhwtJiUy4
2. **Let's Build a WebScraper in 15 Minutes with NextJS!:** https://youtu.be/bGShHOOoC-U?si=FJRuqYa5zPdtYO88